Ln422 Real Time Embedded Systems
Final Assignement
AERO 4 Systems

Student: ALAUX Florian

Date: 28 April 2026 (School Year 2025-2026)

Professor: Mr. Jasdeep SINGH

1 Introduction

This report presents the schedulability of a seven-task set under a non-preemptive policy. The main challenge is to measuring the real Worst Case Execution Time (WCET) of a computational task τ_1 and finding an optimal schedule on an 80s hyperperiod. The implementation includes a non-preemptive scheduler that minimizes total waiting time while allowing task τ_5 to miss its deadline. The missing task is generated in C and the solution is implemented in Python and includes schedulability analysis, scheduling algorithm, and visualization.

2 Hardware Measurement and the Task 1 Investigation

2.1 Initial Failure: The Millisecond Scale

The measurement process began with a simple goal: achieving a execution time in the millisecond range for τ_1. Whether running on Windows or WSL, every attempt failed. Initially, a Python loop calling a C script proved to be far too slow because the overhead of process creation dominated the actual computation.

To refine this, a second version was developed where the iteration loop was moved directly into the C code, with Python calling the executable only once. However, this revealed a new problem: to reach a usable duration of 1-3ms, the numbers to be multiplied had to be astronomically large, and even then, the OS jitter made the measurements inconsistent and unschedulable.

2.2 Final Implementation: Scaling to Seconds

To ensure a stable and schedulable environment, the decision was made to shift the entire system scale to **seconds**. By setting the iteration count to $2 \cdot 10^7$ in the C code, we achieved a workload significant enough to be measured reliably. The final measurement protocol consists of a Python script (*recup_time.c*) executing the C binary (*C1_time.c*) 100 times to extract precise statistical data.

Integrated Loop in C1_time.c

```c
int main(){
    srand(time(NULL));
    long long a;
    for (int i = 0; i < 20000000; i++){
        int x = rand() % 10000000 + 100000;
        int y = rand() % 10000000 + 100000;
        a = (long long)x * y;
    }
    return 0;
}
```

2.3 Final WCET Statistical Results

The following results were obtained after 100 iterations, providing a solid Worst-Case Execution Time (WCET) for our analysis:

WSL Terminal Output

```
-- STATISTIQUES TÂCHE T1 (WSL) --
Min:  0.646354 s
Q1 (25%):  0.656072 s
Q2 (Médiane):  0.662474 s
Q3 (75%):  0.669712 s
Max (WCET mesuré):  0.925961 s
WCET retenu (+20%):  1.111153 s

-- ANALYSE DU SYSTÈME --
Utilisation totale (U): 0.7486
Résultat:  Le système est SCHEDULABLE
```

The results confirmed a schedulable U of 0.7486, with a maximum measured time of $0.925s$, leading to a retained WCET (C_1) of $1.111s$ including a safety margin of 20%.

3 Scheduling Analysis and Simulations

3.1 Utilization and Hyperperiod

As we saw it in the results of the python script just before, the processor utilization factor U is calculated as:

$$U = \sum_{i=1}^{7} \frac{C_i}{T_i} \approx 0.7486 \tag{1}$$

Since $U < 1$, the system is potentially schedulable. The analysis is conducted over a hyperperiod of $80s$, which is the Least Common Multiple of all periods, ensuring a complete cycle of task activations.

3.2 Task Set Overview

With C_1 established at $1.111s$ and the task set analyzed over an $80s$ hyperperiod. The total workload represents $59.889s$ of execution, leaving $20.111s$ of theoretical idle time. This 75% load provides a good margin for non-preemptive scheduling.

Task	**Execution Time (C_i)**	**Period (T_i)**
τ_1	1.111 s (Measured)	10 s
τ_2	3.0 s	10 s
τ_3	2.0 s	20 s
τ_4	2.0 s	20 s
τ_5	2.0 s	40 s
τ_6	2.0 s	40 s
τ_7	3.0 s	80 s

Table 1: Task Set Specifications

4 Scheduling Algorithms and Results

We now use the Python script *scheduler.py* to schedule these seven tasks. We used EDF and SJF logic to compare and minimize the Waiting Time.

4.1 Case 1: Strict Deadline Enforcement (EDF)

The Earliest Deadline First policy was used to verify basic schedulability. Under EDF, the system successfully meets all deadlines, though at the cost of a higher total waiting time.

Q1: EDF Results

```
-- Q1 (EDF Strict) --
Attente totale :  111.112 s
Succès :  True
```

The terminal output confirms that all jobs are completed within the 80s hyperperiod without any overflow.

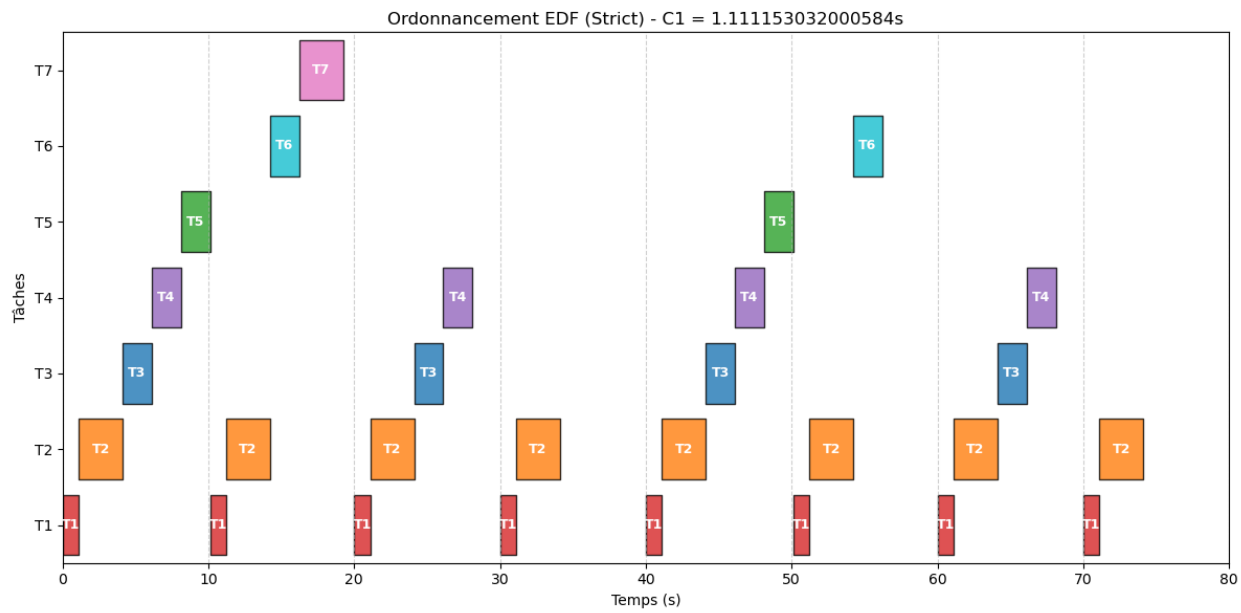

Figure 1: Non-Preemptive EDF Schedule - All deadlines met.

4.2 Case 2: Minimizing Waiting Time (Optimized SJF)

Switching to a Shortest Job First policy reduces the total waiting time from 111.1s to 99.112s. However, a "naive" SJF caused τ_2 to miss its deadlines due to the blocking effect of longer tasks as we can see on the schedule with the red crosses on τ_2 that finished at 10.11s and 50.11s instead of 10s adn 50s.

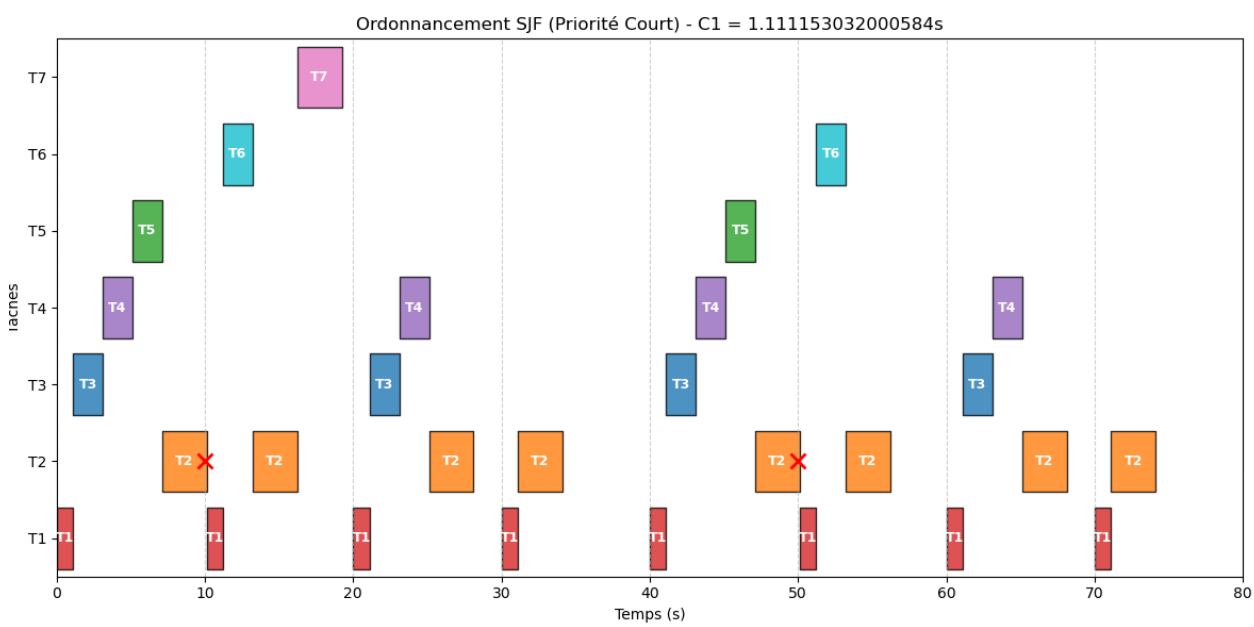

Figure 2: Failure of naive SJF: τ_2 misses its deadline by 0.11s.

To resolve this, we implemented a $3.0s$ protection window, the maximum duration of a single task. If a critical task (other than τ_5) approaches its deadline within this window, it overrides the SJF priority.

Q2: Optimized SJF Results

```
-- Q2 (SJF Optimisé) --
Attente totale :  101.112 s
Retards T5 autorisés :  0
```

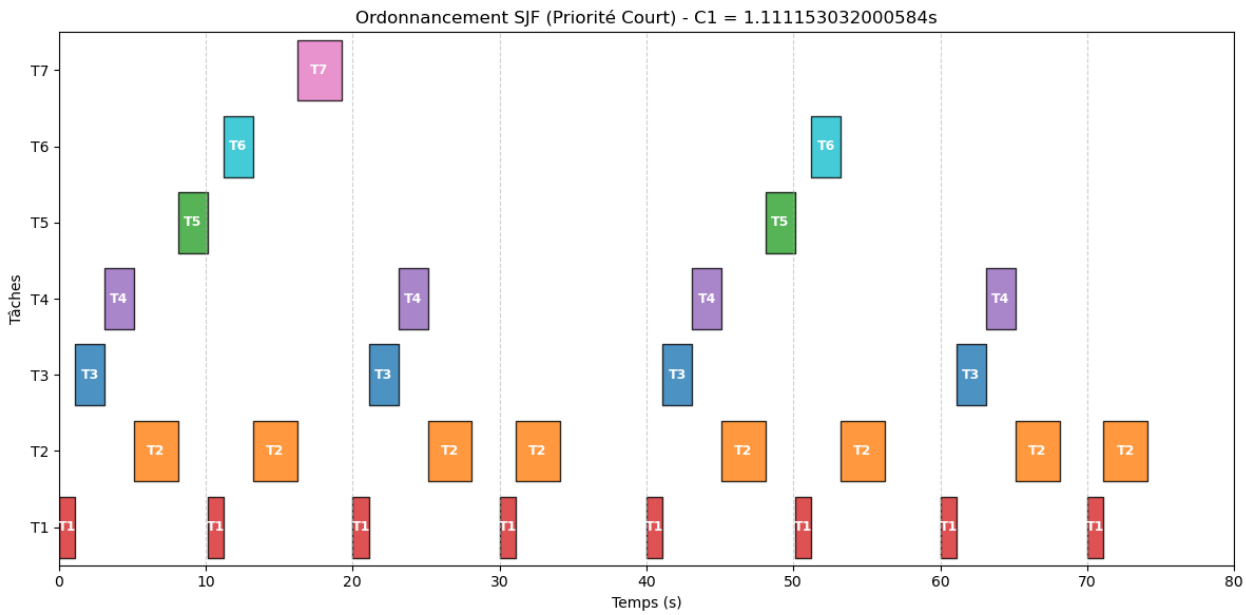

Figure 3: Optimized SJF Schedule: Waiting time minimized without critical failures.

This method works and ends with a total waiting time of $101.112s$, which is $2s$ more than without this window but without missing any deadlines.

5 Response Time and Idle Time Analysis

The response time was verified for each job. As expected, it is minimal for high-priority tasks and increases for lower-priority ones. The Total Idle Time was verified to be $20.111s$ on the $80s$ hyperperiod ($80s - \sum C_i$), confirming that the processor is utilized to its maximum theoretical capacity without any execution leaking into the next cycle.

Idle verification

```
-- MAXIMISATION IDLE VERIFICATION
Somme des exécutions (Travail) :  59.889 s
Idle Time théorique disponible :  20.111 s
Waiting Time total calculé :  111.112 s
Vérification :  SUCCÈS. L'Idle time est de 20.111s.
Aucune exécution ne déborde.
```

6 Conclusion

This study demonstrates the feasibility of scheduling the given seven-task set under a non-preemptive policy. By implementing a simulation over an $80s$ hyperperiod, we identified an optimized priority sequence that satisfies all deadlines. The investigation into Task 1 also revealed that standard process management on an OS like Windows or WSL introduces a level of jitter that invalidates millisecond-scale scheduling. By adapting the workload to a second-scale and implementing an optimized SJF algorithm, we achieved a total waiting time of 101.112s, which is a 10-second improvement in system responsiveness over the standard EDF policy.

Regarding the specific requirements we had in the second part, is important to note that we were explicitly allowed for deadline misses on τ_5 to further reduce latency. But our optimized schedule met all deadlines for every task. This 100% success rate proves the efficiency of the 3.0s protection window. However, this also indicates that our current waiting time is not yet at its theoretical minimum; a more aggressive approach, strictly following the allowed failure of τ_5, could serve as a path to push the system's performance even further by testing every possible priority permutation, similar to a brute-force search, and filtering only the valid sequences that allow τ_5 to fail. By exploring all $n!$ combinations and selecting the one with the lowest latency, we could potentially identify a personalised schedule that reaches the true mathematical limit of the system's efficiency.